Peer Reviews

DTSC 3601: Data Science Studio 3, Predictive Analysis and Their Impacts

Two reviews of Week 5 project drafts

Reviewer: Davis Higgins · Submitted Thursday, September 24, 2026

Both drafts were reviewed against the same five criteria: whether the claim is supported by the evidence presented, whether the evaluation could be trusted by someone who did not run it, whether the writing lets a reader reconstruct what was done, whether the risks of the system are addressed, and whether the work is reproducible. Reviews are anonymised per the course process and are addressed to the authors.

Review 1. Manuscript A

Title: "Nowcasting fine particulate matter in Mecklenburg County from low-cost sensor and weather APIs"

Recommendation: Accept with major revisions. The core work is sound and the engineering is genuinely good. Two problems in the evaluation currently make the headline number unsafe to quote, and both are fixable before Week 7.

Summary of the work

The authors ingest hourly PM2.5 readings from a network of low-cost sensors across Mecklenburg County, join them to National Weather Service observations and forecasts, and predict PM2.5 six hours ahead. Data flows through S3 into Redshift, a random forest and an LSTM are compared, the random forest wins, and the model is served behind an API with a public dashboard showing current and forecast air quality by neighbourhood. They report an RMSE of 3.4 µg/m³ and an R^2 of 0.81 and argue this is good enough to support an early-warning notification for people with respiratory conditions.

What works well

The ingestion layer is the strongest part of this draft, and I do not think the authors give themselves enough credit for it in the writing. The handling of sensor dropout in Section 3.2, where a sensor that has gone silent is distinguished from a sensor reporting zero, is exactly the kind of thing that quietly ruins this class of project, and it is handled properly and documented. The decision to keep the raw API responses in S3 rather than only the parsed rows is correct and will matter the first time a number is questioned.

The choice to compare against a persistence baseline is also right, and I wish more drafts did it. Knowing that "assume PM2.5 in six hours equals PM2.5 now" gets RMSE 4.9 is what makes the 3.4 interpretable at all.

The neighbourhood-level dashboard is a genuinely good product decision. Presenting this at the sensor level would be a false precision that the underlying data does not support, and aggregating up is the honest move.

Major issue 1: the split is random, and it should not be

Where: Section 4.1, "we split the data 80/20 with `train_test_split` and a fixed random seed."

This is the issue that most affects whether the headline number can be believed. PM2.5 is strongly autocorrelated at the hourly scale. A random split puts hour 14:00 in training and hour 15:00 in test, so the model is effectively being asked to interpolate between two readings it has already seen rather than to forecast. The reported R^2 of 0.81 is almost certainly measuring interpolation skill, not forecasting skill, and the gap between the two is usually large.

The persistence baseline does not protect against this, because it suffers from the same problem: a random split flatters persistence too.

How to fix it. Split by time, not at random. Train on the earliest 70 percent of the series, validate on the next 15 percent, test on the most recent 15 percent, with no shuffling anywhere. If the data spans more than a year, prefer a rolling-origin evaluation: train on months 1 to k, test on month k+1, advance, and report the distribution of scores across origins rather than a single number.

I would expect the reported R^2 to fall, possibly substantially. That is not a bad outcome for the paper. A model that forecasts at R^2 0.6 on a proper temporal split is a more valuable and more publishable result than one that interpolates at 0.81, and the paper will be much stronger for saying so directly.

Effort: low, perhaps two hours of rerunning. **Impact on the conclusions:** potentially large, which is exactly why it has to be done before Week 7.

Major issue 2: two features are only available after the fact

Where: Table 2 lists `daily_mean_pm25` and `daily_max_pm25` among the features.

If those are computed over the calendar day containing the prediction timestamp, they include hours that have not happened yet at prediction time, which is leakage. At 09:00 the model cannot know the day's maximum. Given that PM2.5 is autocorrelated and the target is six hours ahead, either feature would carry a large amount of the answer, and I suspect they account for a meaningful share of the reported performance.

I may be wrong. Section 3.4 is not explicit about the window, and it is possible these are computed over the trailing 24 hours rather than the calendar day, which would be fine.

How to fix it. First, state the window explicitly in Section 3.4: "computed over the 24 hours ending at the prediction timestamp" removes all ambiguity. If they are calendar-day aggregates, replace them with trailing-window versions and rerun. Second, and I would recommend this regardless, add a shuffled-label

control: train the full model on randomly permuted targets and confirm the score collapses to chance. It takes one extra training run and it is the cheapest possible insurance against exactly this class of error. I ran one on my own project and it is the single check I would keep if I could only keep one.

Moderate issue: the safety claim outruns the evidence

Where: Section 6, "the system can provide early warning to residents with asthma and COPD."

This is a health claim, and the evaluation does not currently support it. RMSE and R^2 are averaged over the whole distribution, and the whole distribution is dominated by ordinary days. What matters for an asthma warning is performance in the tail: does the model catch the transition into the "unhealthy for sensitive groups" band at 35.5 µg/m³?

An RMSE of 3.4 could easily coexist with the model missing most threshold crossings, because threshold crossings are rare and contribute little to a squared error averaged over thousands of ordinary hours.

How to fix it. Add a section that evaluates the model as the classifier it is being used as. Report, at each Air Quality Index band boundary, the number of true crossings in the test period, how many the model caught, the false alarm rate, and the median lead time on the ones it caught. Two or three sentences and a small table. If the model catches most crossings, that is a much stronger result than the R^2 and deserves to be the headline. If it does not, the paper should say so and the dashboard should stop describing itself as an early-warning system.

Minor points

- **Section 2** cites four papers, three of which are about PM2.5 forecasting with research-grade monitors. The interesting related work for this project is the literature on low-cost sensor calibration and drift, because that is the actual measurement problem here. A paragraph on it would strengthen the framing considerably.
- **Figure 3** uses a rainbow colour map for the concentration heatmap. Rainbow maps are not perceptually uniform and create boundaries in the data that are not there. A single-hue sequential ramp, light to dark, would be both more accurate and easier to read for colour vision deficient readers.
- **Figure 5** has two y-axes with different scales. That plot invites the reader to see a relationship whose apparent strength depends entirely on where the axes were anchored. Two stacked panels sharing an x-axis would say the same thing without the implication.
- **Section 5.2** reports the LSTM result in one sentence and moves on. Given that it lost, it would be worth two more sentences on *why*: was it undertrained, was the sequence length too short, was the sample too small? A negative result with a diagnosis is useful; a negative result without one reads as a box being ticked.
- The model card in Appendix B does not have an out-of-scope section. Given the health framing, it should, and it should say something specific about not using the output for individual medical decisions.

- **Typography:** the RMSE is given as "3.4" in the abstract, "3.42" in Table 3 and "3.4187" in Section 5.1. Pick a precision and hold it. Two significant figures beyond the decimal is plenty here.

Summary for the authors

The engineering under this paper is better than the evaluation currently lets you show. Fix the split, resolve the aggregate-window question, and add a threshold crossing analysis, and you will have a result you can defend hard in the presentation. All three are achievable in the time remaining. I would rather read the honest lower number with a threshold analysis behind it than the current 0.81.

Review 2. Manuscript B

Title: "A risk-scoring pipeline for prioritising restaurant health inspections in Mecklenburg County"

Recommendation: Accept with revisions. The modelling is careful and the writing is clear. My concerns are about what the system does once deployed rather than about whether the model works, and I think addressing them would make this the strongest project in the section.

Summary of the work

The authors pull ten years of Mecklenburg County restaurant inspection records through the county's open data API, join in business licence data and weather, and predict whether an establishment's next inspection will produce a critical violation. XGBoost with SMOTE reaches an F1 of 0.62 and a ROC AUC of 0.78 on a held-out set. The output is a ranked list intended to help the county schedule inspections, on the argument that inspecting high-risk establishments sooner finds problems earlier.

What works well

The framing is excellent. This is a genuine resource allocation problem with a real constraint (a fixed number of inspector-days), and the paper is clear about the decision the model is meant to inform rather than treating prediction as an end in itself. Section 1 is the best-written introduction I read in this exchange.

The temporal split is done correctly, and Section 4.1's explanation of why is worth keeping verbatim. So is the decision to hold out entire establishments rather than individual inspections, which prevents the model from memorising a specific restaurant's history and then being tested on it. That is a subtle point and the authors clearly thought about it.

The feature engineering around inspection history is thoughtful, particularly `days_since_last_inspection` and the rolling count of prior critical violations.

Major issue 1: SMOTE is probably hurting you, and it is definitely hurting your

probabilities

Where: Section 4.3, "we applied SMOTE to address the 18 percent positive class rate."

Two problems.

The first is a bug risk. Section 4.3 does not say whether SMOTE was applied before or after the split. If before, synthetic points interpolated from test-set neighbours have leaked into training, and the reported scores are inflated by an unknown amount. The text is ambiguous and this needs one explicit sentence either way. If it was applied inside a pipeline fitted only on training folds, say so plainly, because a reader currently cannot tell.

The second is more fundamental. SMOTE changes the class balance, which changes the predicted probabilities, which means the model's output no longer corresponds to any real frequency. That does not matter if you only care about the ranking. It matters a great deal for this application, because Section 6 proposes showing inspectors a risk score, and an inspector will read 0.7 as "seven in ten". After SMOTE it does not mean that.

Also worth knowing: an 18 percent positive rate is not severe imbalance. XGBoost handles it natively through `scale_pos_weight`, and in my experience on a similarly imbalanced target, resampling bought nothing that the built-in weight did not.

How to fix it. Run three configurations and report all three: no resampling, `scale_pos_weight`, and SMOTE. If they perform comparably, drop SMOTE and keep the interpretable probabilities. Then calibrate the winner with isotonic or Platt scaling on a held-out block and report the expected calibration error alongside the F1. A calibration plot is one figure and it would materially strengthen the paper.

Major issue 2: the target may encode inspector behaviour rather than restaurant

risk

Where: Section 3.1, target definition.

This is the concern I would most want the authors to engage with, and I raise it because I think it is interesting rather than because I think it is fatal.

The label is "a critical violation was recorded". That is the output of an inspection, and inspections are conducted by people who differ in strictness, whose route assignments are not random, and whose visit frequency is already determined partly by an establishment's history. If some inspectors write more critical violations than others, and if inspectors are assigned geographically, then the model may be partly learning which inspector covers which part of the county.

That would still produce a model that predicts recorded violations accurately. But deploying it would concentrate future inspections in the areas covered by stricter inspectors, which would generate more recorded violations there, which would reinforce the pattern. This is structurally the same mechanism Obermeyer et al. (2019, *Science* 366:447-453) describe in healthcare: a model accurate at predicting its proxy, harmful because of what the proxy encodes.

How to check it. If the data carries an inspector identifier, add it as a feature and see how much it moves the AUC. If it moves it a lot, that is the finding. If it is not available, a geographic breakdown of performance and of predicted risk by ZIP code would surface the same pattern indirectly. Either way, one paragraph acknowledging the concern would substantially improve the paper's credibility, and if you find the effect is small you get to say so with evidence.

Moderate issue: F1 is not the metric this application needs

Where: Section 5, headline result.

F1 collapses precision and recall at one threshold, and the threshold is not stated. More importantly, the county does not choose a threshold; it has a number of inspector-days and works down a list.

How to fix it. Report precision at *k* for the values of *k* the county actually operates at. If inspectors can do 200 inspections a month, report precision at 200 and the lift over the current scheduling policy. That single number is the one a county official would ask for, and it is far more persuasive than an F1. It is also easy: you already have the ranked scores.

Moderate issue: no monitoring section

The course brief asks for a pipeline that notices when the data or the model has changed, and Section 7 currently describes the CloudWatch alarms for Lambda errors but nothing about the model. Restaurant inspection is exactly the domain where this matters: the county changed its inspection scoring rubric in 2020, and if that change is inside your data, your target definition is not stable across the series.

How to fix it. Two additions. First, check whether the rubric change is in your window and, if so, whether the positive rate shifts at that date; a monthly plot of the base rate would show it immediately. Second, add a drift check on the input features, population stability index against the training block is fine, and a rolling performance metric against a baseline established on your validation block. This does not need to be elaborate to satisfy the brief and to be genuinely useful.

Minor points

- **Table 1** reports feature importances from XGBoost's default `gain`. Gain is biased toward high-cardinality features and says nothing about direction. Permutation importance on the held-out set answers the question you are actually asking, and SHAP would give you the direction as well.

- **Section 5.4** says the model "identifies" high-risk restaurants. It predicts recorded violations. The distinction matters for exactly the reason in Major issue 2, and the more careful verb costs nothing.
- **Figure 2** plots the ROC curve without the baseline diagonal. Add it.
- The related work section covers three papers on food safety prediction but none on the general problem of predictive allocation of inspections, which has a literature in tax audit and building-code enforcement with directly transferable lessons about feedback loops.
- Section 3.2 mentions dropping establishments with fewer than three inspections. How many were dropped, and are they systematically different? New businesses are plausibly higher risk, and if they are being excluded, the deployed model would never rank them.
- **Reproducibility:** the paper does not state a random seed or the XGBoost version. Both should be in the appendix.

Summary for the authors

This is careful work on a well-chosen problem, and the temporal and establishment-level splitting shows real attention to the things that go wrong. My substantive concerns are about deployment rather than modelling: replace SMOTE with a class weight so the probabilities mean something, report precision at the county's actual capacity instead of F1, engage with the inspector-effect question even briefly, and add a monitoring section. The first three are a few hours of work. The fourth is required by the brief and would also, I think, be the most interesting section in the paper once written.

A note on my own bias as a reviewer

Both reviews lean hard on evaluation design, leakage and calibration, and I should say why: those are the three things I spent most of my own project on, so they are the things I notice. Neither author should read the length of a comment as a measure of how serious the problem is. The two issues I have marked Major in each paper are the ones I would actually prioritise; everything under Minor is genuinely minor and both drafts would be respectable submissions if only the Major items were addressed.